Smart Industrial Safety Monitoring System - Project Report

Members :

- **Pranav Sharad Nasne** (Registration No - 24BEC045)
- **Kasuhik Ashok Naik** (Registration No - 24BEC028)
- **Soham Rajendra Nalage** (Registration No - 24BEC029)
- **Rajkumar Sarraf** (Registration No - 24BEC050)
- **Nishit Gupta** (Registration No - 24BEC034)

Course Instructor - Dr. Prakash Pawar

Department of Electronics and Communication Engineering

Abstract

This project focuses on enhancing industrial workplace safety by deploying an automated monitoring system. Utilizing a Raspberry Pi integrated with an array of environmental and physical sensors (Vibration, Gas, Flame, and DHT11), the system tracks real-time data to detect hazardous conditions. Since the Raspberry Pi lacks native analog pins, an ADC (Analog-to-Digital Converter) is employed to interface with analog sensors. The data is visualized on a web-based dashboard, providing instant alerts when sensor values don't follow ML model.

Hardware Specifications

To replicate or understand the system architecture, the following components are utilized:

- **Microcontroller:** Raspberry Pi
- **ADC Module:** MCP3008 (to convert analog sensor signals for the Pi).
- **Sensors:**
    a) **Vibration Sensor:** Detects structural instability or machine malfunction.
    b) **Gas Sensor (MQ Series):** Monitors leakage of combustible gases or smoke.
    c) **Flame Sensor:** Identifies the presence of fire or high-intensity infrared light.
    d) **DHT11:** Measures ambient temperature and humidity levels.

- **Connectivity:** Ethernet/Wi-Fi for web server communication.

Software Stack

- **Frontend: React.js** – Used for building a dynamic, responsive user interface. It allows for real-time dashboard updates without refreshing the page.
- **Backend: Flask (Python)** – A lightweight WSGI web framework used to create APIs that bridge the Raspberry Pi sensor data and the React frontend.

Machine Learning Model

- **Random Forest Classifier**: A supervised ensemble learning model used for multi-class hazard classification.
- **Isolation Forest:** An unsupervised anomaly detection algorithm used to identify "unknown" dangerous patterns that don't fit into pre-defined categories.

Methodology: The Intelligent Decision Engine

Dual-Layer Logic Architecture

To ensure 100% reliability, the system operates on a prioritized logic flow:

1. **Primary Layer (ML - Random Forest):** The system first passes the normalized sensor data through a trained Random Forest model. This model analyzes the relationship between variables.
2. **Fail-Safe Layer (Threshold Heuristics):** In the event of a model "Low Confidence" score or a script error, the system defaults to hardcoded Threshold Logic. This ensures that even if the AI is uncertain, a simple physical breach (like Temperature > Threshold) will always trigger the alarm.

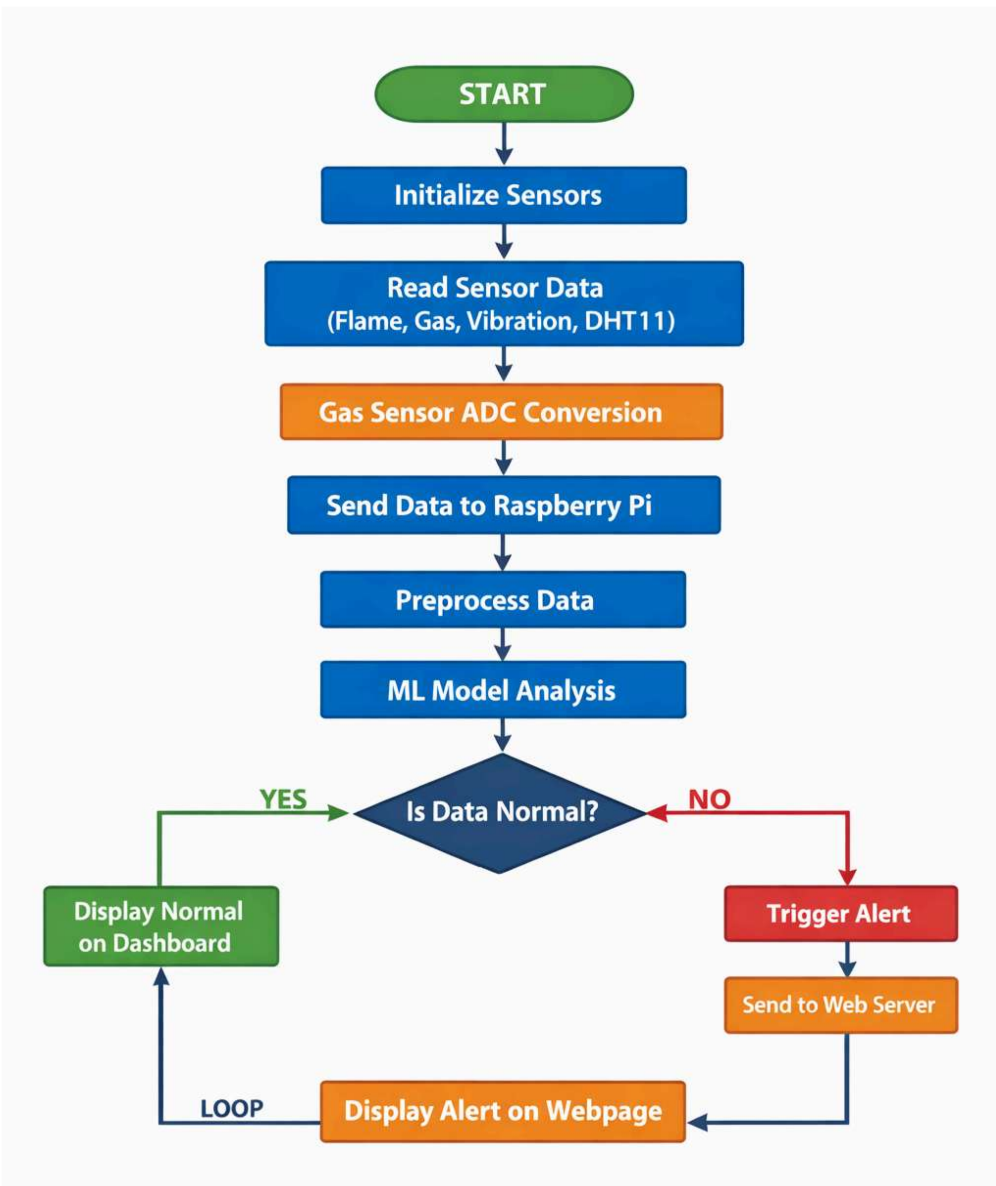
System Workflow

To visualize the operation from physical sensing to the cloud-based alert, follow this logical flow:

1. **Data Acquisition:** Sensors (Flame, Vibration, Gas, DHT11) continuously poll the environment for raw physical data.
2. **Signal Conversion:** The MCP3008 ADC converts analog signals from the MQ Gas and Flame sensors into 10-bit digital values.
3. **Local Processing:** Raspberry Pi receives digital data via SPI (for ADC) and GPIO (for DHT11/Vibration).
4. **Intelligence Layer:**
    - **ML Analysis:** Data is passed through the **Random Forest Classifier** to identify specific hazard types.

    ○ **Anomaly Detection:** The **Isolation Forest** checks for patterns that don't match standard safety or known danger profiles.

5. **Fail-Safe Check:** If the ML model has low confidence, the system checks values against hardcoded safety thresholds.
6. **Web Communication:** The Flask backend serves the processed data via an API.
7. **Real-Time Visualization:** The React.js frontend fetches API data to update the "NEXUS" dashboard and trigger alerts if a breach is detected.



System Explanation

The system consists of multiple sensors including Flame, Gas, Vibration, and DHT11 connected to a Raspberry Pi. Since the gas sensor produces analog output, an ADC is used to convert it into digital form.

The Raspberry Pi continuously collects sensor data and processes it in real-time. A Machine Learning model is used to analyze the data and identify abnormal patterns instead of relying on fixed thresholds.

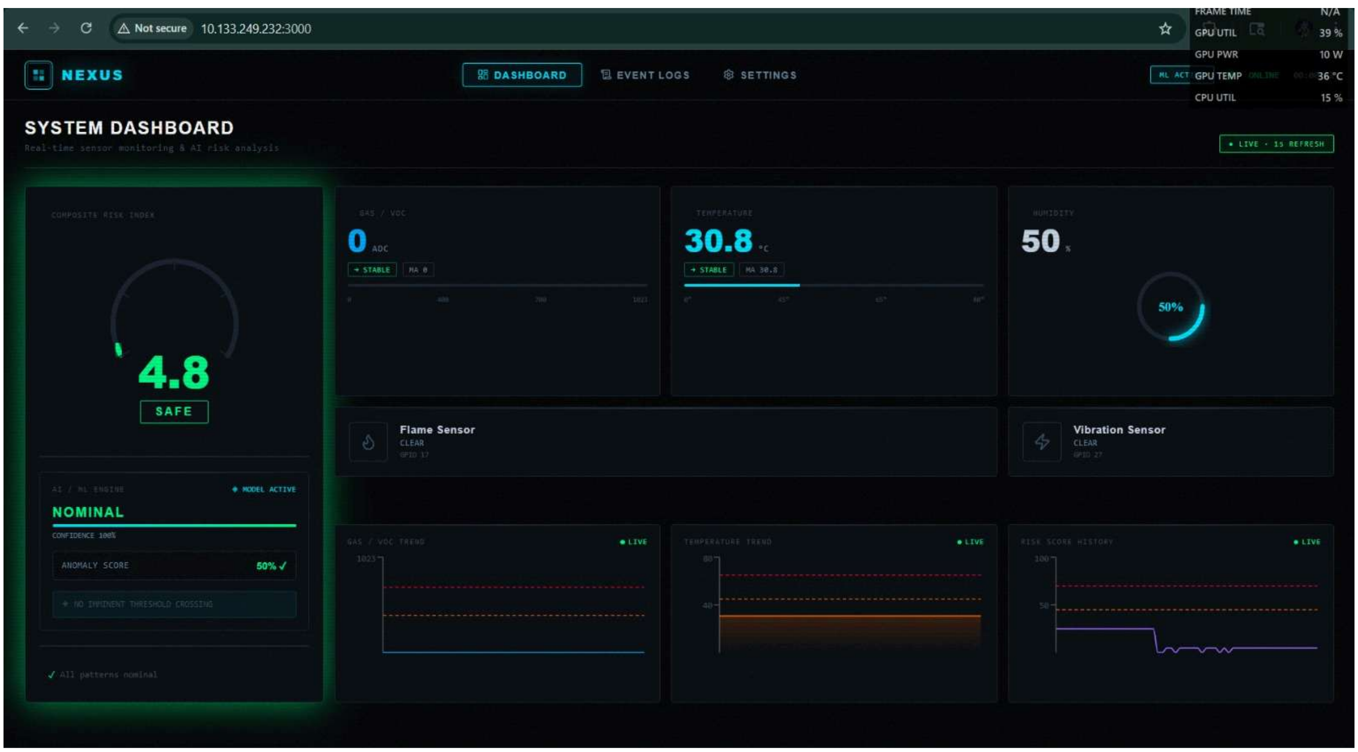
When any anomaly is detected, the system generates an alert which is displayed on a web dashboard. This enables real-time monitoring and quick response to potential hazards.

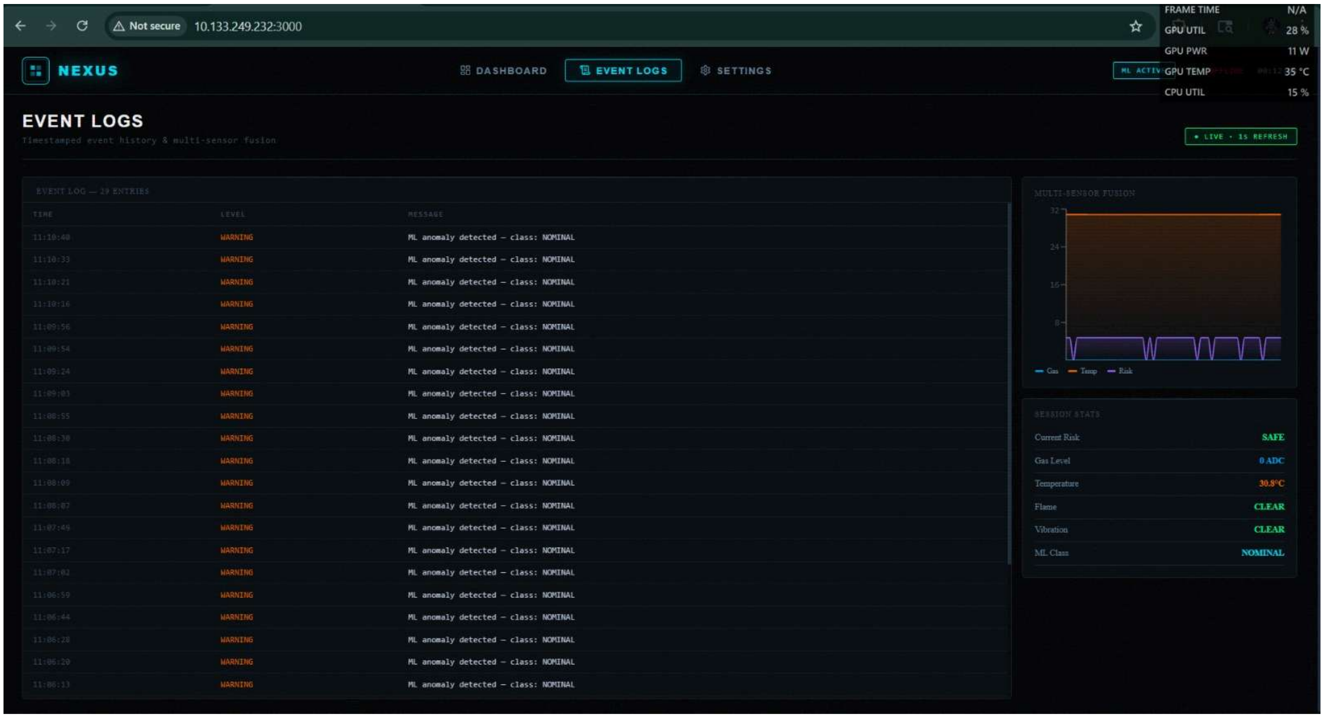
Wiring Summary

COMPONENTS	INTERFACE TYPE	CONNECTION DETAILS
MCP3008	SPI Communication	Connects to Pi's SPI pins (MOSI, MISO, SCLK, CE0). It provides 8 channels for analog inputs.
Gas Sensor	Analog	Connected to **CH0** of the MCP3008.
Flame Sensor	Digital/Analog	Connected to **CH1** of the MCP3008 for intensity readings.
Vibration Sensor	Digital	Connected directly to a GPIO pin to detect high/low pulses from mechanical stress.

DHT11	Digital	Connected to a GPIO pin with a pull-up resistor to transmit Temperature/Humidity data
Network	Ethernet	Connects the Pi to the local network to host the Flask server and React dashboard.

Experimental Results & Dashboard





Features

- Real-time monitoring
- Multi-sensor integration
- ML-based anomaly detection
- Web-based alert system
- Scalable architecture

Applications

- Industrial safety systems
- Smart homes
- Fire detection systems
- Gas leakage monitoring
- Earthquake/vibration detection

Future Enhancements

To evolve this prototype into a high-end industrial product, the following features can be integrated:

- **Automated Visual Verification:** Integration of a PiCamera module. Upon a threshold breach, the system will trigger an image capture or a short video clip of the site and send it via email or Telegram bot to the safety officer.
- **Gyroscope Integration** : If there is any misplacement in the machines or the system it will trigger the alert that the stability of the system is compromised.

Conclusion

The **Smart Industrial Safety Monitoring System** provides a robust, low-cost solution for real-time hazard detection. By bridging the gap between hardware sensors and web-based monitoring, it ensures that industrial supervisors can maintain a safe environment from any remote location, significantly reducing the response time during accidents.
